Redfish Aliasing support and Telemetry enhancements

Jeff Autor (Vertiv), Hari Ramachandran (Microsoft),
Bill Scherer (HPE)

Redfish Forum, October 2026

Copyright © 2026 DMTF

Redfish

www.dmtf.org

Disclaimer

- The information in this presentation represents a snapshot of work in progress within the DMTF.
- This information is subject to change without notice. The standard specifications remain the normative reference for all information.
- For additional information, see the DMTF website: www.dmtf.org

Problem: Hypermedia discovery becomes a fleet tax

- Discovery cost – vendors use different member *Id* naming logic
  - Requiring adoption of customer-specific naming rules is problematic
- Must inspect properties to infer physical / logical “role”
  - Example: Which NIC is “northbound” vs “east-west”
  - Usage cannot be determined by firmware alone without end user input
- OCP and hyperscalers raised this issue to DMTF / Redfish Forum
  - Various proposals discussed that would retain compatibility
  - Group converged on a user-defined “resource alias” concept
  - Concept was approved and added to Redfish Specification
    - Entire process occurred within a single release cycle (3-4 times per year)

NEW Resource Aliases

- Added in Redfish Specification v1.25.0 and schema release 2026.2
- Resource aliases provide user-assigned, implementation-independent names for members of resource collections
  - Allows access to a resource without discovering its *@odata.id* or *Id* value
- Clients can address resource using alias(es) prefixed with a tilde "~"
  - `/redfish/v1/Systems/~MySystem/Processors/~MyGPU`
- **NEW** Writable *@Redfish.Alias* annotation contains the configured alias
- Alias values:
  - Cannot contain URI-unsafe or reserved characters, require percent-encoding, or the tilde character
  - Must be unique, both within collection and across collections of same schema type with a common parent
  - Can be reassigned to another resource; previous resource's alias is cleared and the service emits the *AliasValueReassigned* message from the Base Message Registry

Example round-trip I/O improvement

Without Aliases: Requires Discovery

1.
```
systems = GET(host, "/redfish/v1/Systems")
system_uri = systems["Members"][0]["@odata.id"]
```
2.
```
processors = GET(host, f"{system_uri}/Processors")
processor_uri = processors["Members"][0]["@odata.id"]
```
3.
```
data = GET(host, processor_uri)
return data["Status"]["Health"]
```

With Aliases: **No discovery required**

1.
```
data = GET(host, auth, "/redfish/v1/Systems/~system1/Processors/~cpu1")
return data["Status"]["Health"]
```

Telemetry Support

- New Telemetry Task Force formed in April 2026
  - Dedicated team to drive interoperable telemetry topics for Redfish
- Addressing known limitations:
  - Define standard, interoperable names and labels
  - Enable setup of telemetry output without a priori knowledge
  - Increase adoption by telemetry client ecosystem
- Leveraging *Alias* concept became first order of business

NEW ServiceAlias for telemetry labels (in progress)

- For interoperability, create standards-based *Alias* values
  - Operate alongside user-defined *Alias* using the same mechanisms
- *@Redfish.ServiceAlias* contains a service-owned, standard alias
  - Read-only value with unique pattern defined for each schema
  - Entire URI path can be recovered from label values alone
- These are essentially the "hard-coded *Id* values" everyone wants...
  - `GET /redfish/v1/Systems/~System1/Processors/~CPU0`
- The *ServiceAlias* values provide consistent, interoperable labels
  - "Cache hit rate" value from: `System1, CPU0, Core3`
  - "Power" value from: `RackPDU2, Outlet8`

ServiceAlias construction

- Constructed of schema name (or abbreviation) and ordinal value
  - Unique naming pattern to allow recovery of URI segments
  - Must include an ordinal, with ordering rules defined in schema
- May include a schema sub-type, typically based on a key property
  - **CoolingUnit** *EquipmentType*: `CDU, ImmersionUnit, RPU`
  - **Processor** *ProcessorType*: `CPU, GPU, Core, Thread`
  - **Sensor** *ReadingType*: `Temperature, Power, Current`
- Ordinal values are sequential and incremental
  - Values are 1-based to reflect physical or simple ordering (first, second)
  - Some 0-based exceptions for **Processor** and **Drive**

ServiceAlias patterns

- **AutomationNode**: AutomationNode1
- **Battery**: Battery1
- **Chassis**: Chassis1
- **Circuit**: <CircuitType>1 Branch, Mains
- **ComputerSystem**: System1
- **Control**: <ControlType>Control1
  - Temperature, Humidity, PressurekPa
- **CoolantConnector**: <Category>Connector1
  - PrimaryConnector, SecondaryConnector
- **CoolingLoop**: CoolingLoop1
- **CoolingUnit**: <EquipmentType>1
  - CDU, RPU, HeatExchanger
- **Drive**: Drive0
- **EthernetInterface**: Ethernet1
- **Fabric**: Fabric1
- **FabricAdapter**: FabricAdapter1
- **Fan**: Fan1
- **Filter**: Filter1
- **Heater**: Heater1
- **Manager**: Manager1
- **Memory**: <Category>1 Memory, GPUMemory
- **NetworkAdapter**: NetAdapter1
- **Outlet**: Outlet1
- **OutletGroup**: OutletGroup1
- **PCIeDevice**: PCIeDevice1
- **Port:** Port1
- **PowerDistribution**: <EquipmentType>1
  - RackPDU, FloorPDU, PowerShelf
- **PowerSupply**: <PowerSupplyType>1 PSU, VoltageRegulator
- **Processor**: <ProcessorType>0 CPU, GPU, Core, Thread, FPGA
- **Pump**: Pump1
- **Reservoir**: Reservoir1
- **Sensor**: <ReadingType>1 Temperature, Power, Voltage
- **Storage**: Storage1
- **StorageController**: StorageCntr1
- **Switch**: Switch1
- **Volume**: Vol1

Interoperable Telemetry Name and Label examples

- Redfish **Sensor** model supports bulk of infrastructure telemetry
  - `Sensor_Temperature : CDU1, PrimaryConnector1, Supply`
  - `Sensor_Temperature : CDU1, PrimaryConnector1, Delta`
  - `Sensor_Power : PDU2, Outlet4   (per outlet power)`
  - `Sensor_Power : PDU2, Branch3, Line1 (per conductor power)`
  - `Sensor_Power : System1    (total server power)`
  - `Sensor_Power : System1, GPU0 (per processor power)`
- Processor, Memory, Networking models (objects) flattened
  - `ProcessorMetrics_CacheHitRatio : System2, CPU0, Cache0`
  - `ProcessorMetrics_CacheHitRatio : System2, CPU1, Core4, Cache0`
  - `MemoryMetrics_BlocksRead : System1, Memory3, CurrentPeriod`
  - `MemoryMetrics_BlocksWritten : System4, Memory2, LifeTime`

Interoperable Names and Labels Construction

- "Telemetry Name": Data point being measured
  - Unique names already exist: "`Schema [+ Path] + Property`"
  - Added text replacement methods to improve results (used sparingly)
- "Telemetry Label": Which instance of the data point is measured
  - Concatenate *ServiceAlias* values
  - Option to utilize user-supplied *Alias* values as override
  - Append in-resource disambiguation when needed (i.e. arrays)
- This combination provides interoperable telemetry
  - Value type, units, format, etc. – are already schema-backed

“TelemetryBundle” output proposal

- Supports time series (TLV) output format
  - One “metric” per record, optional bundle of records delivered together
- Optimized payload size compared to existing **MetricReport**
  - Bundles can share time, name and/or label
  - Intended as drop-in replacement with interoperable names and labels
- Specify “metric name” and “metric label” string formats
- Bundling of records:
  - Same timestamp, multiple sensors: “16 temperature readings”
  - Time series, same sensor: “Pressure every 5 seconds over 10 min”
  - Both…

NEW TelemetryBundle proposal

```
{
   “Time”: 1743984255,
   “Name”: “Temperature”,
   “Schema”: “Sensor”
   “Labels”: [ “System1”, “CPU0” ],
   “Context”: “User-supplied context…”,
   “Records”: [{
      “Time": 1743984253.25,
      “Schema”: “Sensor”
      “Name”: “Temperature”,
      “Labels”: [ “System1”, “CPU1” ],
      "Value": “42.1”,
      },
      . . . More telemetry data ...
   ]
}
```

Adopt epoch-style timestamps (bundle plus per-record)

Single-record style uses TLV properties at root
Time, Name, and/or Label can optimize bundle content
(omit when same across all records in bundle…)

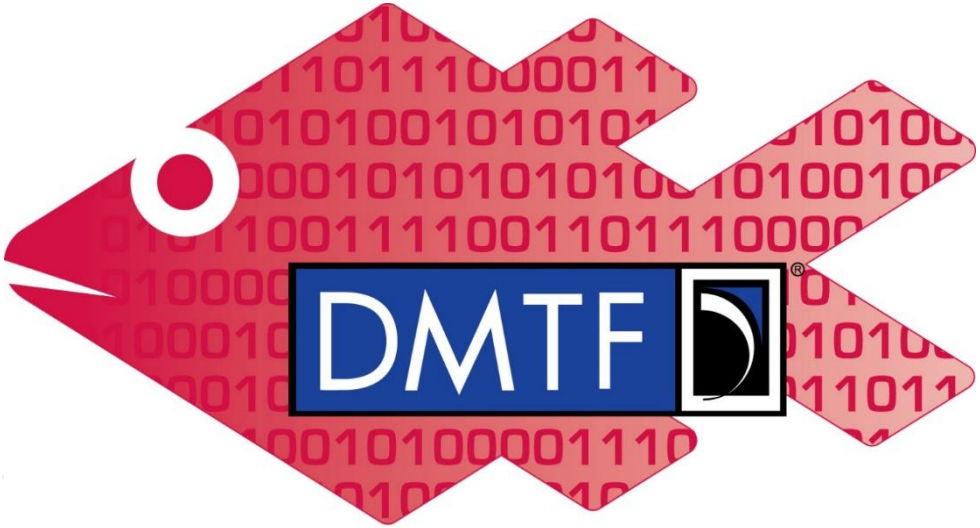
Multiple-record style can omit “shared”
Schema, Name, Time and/or Labels values
(use root value) for efficiency

Redfish-defined interoperable Name and Label,
Constructed from schema name, property name,
and ServiceAlias values.

Telemetry Task Force – Work in Progress topics

- Resource Subset-based selection and output
  - Deliver “telemetry-only” resource subsets
  - Intended to provide an easier method to define a stream
    - Specify by resource (schema) instead of per-property
  - More efficient telemetry format for many use cases
    - But this requires education, TLV is the de facto answer…
- **TelemetryStream** – concept for stream subscriptions
  - Tailored specifically for managing streaming telemetry subscribers
  - Would also enable generation of “reports” (local or remote)

Q&A & Discussion



Redfish